iPRA without an introduction witness parameter

A consequence-equivalent restriction of the native scalar calculus

Research report prepared for Vladimir Reshetnikov

12 November 2026

Abstract

An existential introduction normally supplies an arbitrary numerical term. In the native scalar presentation of intuitionistic primitive recursive arithmetic, that argument can be removed entirely: always use the newest ambient numerical parameter, and use 0 when the ambient scope is empty. All native consequences remain derivable, including consequences of arbitrary open assumptions. The construction applies both to the eight-rule kernel and to its seven-rule diagonal variant. Its ordinary realization is therefore bi-interpretable with full first-order iPRA, with whole numerical domains, actual equality, and the same identity numerical round trips as the current proposal.

The essential ingredients are a predecessor construction using equational induction alone, a uniform naming formula obtained by numerical cases, and a proof translation that regenerates premises after extending the parameter scope. The last point avoids assuming that a completed proof with a scope-dependent witness can simply be renamed or weakened.

The archive includes complete standalone Python checkers and matched controls. In the supplied numeric syntax the seven-rule checker decreases from 932/1029/2770 to 924/1021/2753 significant tokens / AST nodes / normalized bytes. This is a modest, reproducible implementation reduction supporting a new mathematical restriction. It is not a new global byte record and is not a proof of absolute minimality. The new equivalence is proved here in written mathematics; it has not been checked in Lean.

1 Result and comparison target

The comparison is pinned to commit

`9f9545f5ffd9e0392d4a36639503aca12383450a`

of `VladimirReshetnikov/ProveIt`, in `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The project already contains three-family clause calculi, computation normalization, inferred witnesses at elimination, and Lean equivalence theorems [1, 2, 3]. These are established parts of the baseline.

The new restriction concerns the *introduction* witness. Let $\Pi = (a_1, \ldots, a_m)$ be the ordered list of numerical parameters currently in scope, and put

$$\nu_\Pi = \begin{cases} a_m, & m > 0, \\ 0, & m = 0. \end{cases}$$

A fresh eigenparameter is appended to this list. The context Γ is a separate list of assumptions: its last assumption need not introduce the newest parameter.

Theorem 1 (No witness argument)**.** *Let $\mathcal{N}$ be the current eight-rule native kernel. Let $\mathcal{F}$ have exactly the same syntax and rules, except that DI by cases must use $w = \nu_\Pi$, with no witness supplied by the proof. For every finite scope Π, native context Γ, and native formula A in that scope,*

$$\Pi;\Gamma \vdash_{\mathcal{F}} A \quad \Longleftrightarrow \quad \Pi;\Gamma \vdash_{\mathcal{N}} A.$$

Let $\mathcal{G}$ be the current seven-rule diagonal calculus, and let $\mathcal{J}$ make the same deterministic-witness restriction in its single introduction. Then

$$\Pi;\Gamma \vdash_{\mathcal{J}} A \quad \Longleftrightarrow \quad \Pi;\Gamma \vdash_{\mathcal{G}} A \quad \Longleftrightarrow \quad \Pi;\Gamma \vdash_{\mathcal{N}} A.$$

The formula translations in these equivalences are literally the identity. No restriction to a protected public sublanguage, quotient equality, normal-form terms, or selected class of contexts is involved.

Corollary 2. *The ordinary realizations of $\mathcal{F}$ and $\mathcal{J}$ are equiconsistent and bi-interpretable with full first-order* iPRA*. The interpretations use the entire numerical domains and actual equality, and the numerical composites are the identity graph $x = y$.*

This proves the weaker zero-or-variable introduction restriction posed as Question 2 in the pinned README: the deterministic rule uses only those witnesses. The improvement is that the certificate need not choose even which variable to use.

2 Complete presentation

2.1 Syntax and intended readings

There is one numerical sort. Raw terms are

$$t ::= a \mid 0 \mid S(t) \mid I(t, t; u.t),$$

where only u is bound in the last child of I. Formulas are

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

The designated contradiction is $0 = S0$. There are no implicit ordinary logical connectives or hidden structural inference rules. In the ordinary realization the binders mean

$$Q_y B(y) \equiv B(0) \to \forall y\, B(y), \qquad D_{x,y} B(x,y) \equiv \exists x \big(B(x,0) \to \forall y\, B(x,y)\big).$$

These readings establish soundness and compose the bi-interpretations. They are not extra rules of the checker.

All substitutions below are capture-free. Bound letters can be renamed. A term template $h(v)$ has one distinguished free hole; its other parameters are retained.

2.2 The eight-rule presentation $\mathcal{F}$

The following is the complete primitive inventory. Freshness in DE and either introduction means absence from the ambient scope, context, conclusion, and all displayed outside parameters.

Rule	Premises and conclusion
Hyp	If $A \in \Gamma$, infer $\Gamma \vdash A$.
$I0$	Infer $\Gamma \vdash I(0, a; u.h) = a$.
IS	Infer $\Gamma \vdash I(Sc, a; u.h) = h[I(c, a; u.h)/u]$.
Tr	From $\Gamma \vdash s = t$ and $\Gamma \vdash h(s) = r$, infer $\Gamma \vdash r = h(t)$.
QE	From $\Gamma \vdash Q_y B(y)$ and $\Gamma \vdash B(0)$, infer $\Gamma \vdash B(t)$, for any term t in scope.
DE	From $\Gamma \vdash D_{x,y}B$ and $\Pi, b; \Gamma, Q_y B(b, y) \vdash A$, infer $\Pi; \Gamma \vdash A$, where b is fresh.
DI	From $\Pi, z; \Gamma, B(\nu_\Pi, 0) \vdash B(\nu_\Pi, Sz)$, infer $\Pi; \Gamma \vdash D_{x,y}B$, where z is fresh and ν_Π is computed in the *parent* scope Π.
QI	For a literal equation $E(y)$, from $\Pi, z; \Gamma, E(z) \vdash E(Sz)$, infer $\Pi; \Gamma \vdash Q_y E(y)$, where z is fresh.

DI is supplied a matrix and a premise derivation, but no numerical term. QE still permits arbitrary targets; the theorem does not restrict both term arguments simultaneously.

Equality reflexivity is derived. For example $I(0, t; u.u) = t$ transported along itself with the identity template gives $t = t$. Symmetry, transitivity, and term congruence follow from Tr.

2.3 The seven-rule presentation $\mathcal{J}$

Retain Hyp, $I0$, IS, Tr, QE, and DE. Replace the two introductions by one. Inspect the raw matrix $B(x, y)$, before any witness is inserted:

$$\mathrm{diag}(B) \quad \Longleftrightarrow \quad B \text{ is literally } I(y, y; u.h) = I(y, y; v.k).$$

The displayed occurrences of y must be the bound second coordinate, not a free parameter with the same printed name. Set

$$\eta_B(z) = \begin{cases} z, & \mathrm{diag}(B), \\ 0, & \text{otherwise.} \end{cases}$$

The sole introduction is

$$\frac{\Pi, z; \Gamma, B(\nu_\Pi, \eta_B(z)) \vdash B(\nu_\Pi, Sz)}{\Pi; \Gamma \vdash D_{x,y}B(x, y)}. \tag{DI*}$$

The matrix alone selects cases or progress; the scope alone selects the witness. This is the existing diagonal introduction with its witness argument deleted. It introduces no new guard, axiom, normalizer, or auxiliary syntax.

3 Avoiding an invalid structural argument

A proof containing deterministic DI cannot in general be weakened to a larger parameter scope merely by copying its tree. If a newer parameter is added, the witness selected at an old DI node changes. Thus naming the desired term and then silently weakening the old proof is insufficient.

We use proof-generating families. Fix a parameter scope and retained assumptions. A *portable proof family* for A is a procedure which, for every finite extension by fresh parameters and additional assumptions, constructs a derivation of the unchanged A in the larger scope. Retained assumptions are realized by such families. Hypothesis lookup, computation, and equational rules construct families directly. No claim about moving a previously completed target proof is part of this definition.

The completed constructions below take portable premises and return portable results: in a larger scope they are run again, choosing fresh local eigenparameters there. The final translation inducts on a *source* derivation, under an arbitrary substitution of its free parameters and portable realizations of its hypotheses. It returns a portable target proof family. Ordinary assumptions have such realizations by Hyp.

A local case proof whose witness is a freshly chosen z is invoked while z is newest. It is *not* assumed portable with that fixed z. In any larger outer scope, the complete enclosing construction chooses a new case parameter last. This distinction is used explicitly in naming.

3.1 Cases and full formula transport

The derived rules in this section transform the proof-generating families just defined. Supplying a derived base means supplying its portable realization, not applying an unproved cut rule to an arbitrary completed target proof.

In $\mathcal{F}$, conditional closure by cases is available for every formula:

$$\frac{\Gamma, B(0) \vdash B(Sz)}{\Gamma \vdash Q_y B(y)}. \tag{QC}$$

Introduce $D_{r,y}B(y)$, with r unused, at the automatically selected witness ν_Π. Eliminate it at a fresh parameter and return the opened Q-assumption. The case-premise constructor is run with z fresh and newest in each requested outer scope.

In $\mathcal{J}$, the same construction works. If the matrix is diagonal, $B(\nu_\Pi, 0)$ is a theorem: both sides reduce by $I0$ to 0. Supply that theorem to the case premise and ignore the new progress hypothesis. Otherwise DI* is already in case mode. Thus QC is available before general equation induction is recovered.

Portable full formula transport

$$s = t, \quad F(s) \quad \vdash \quad F(t) \tag{L}$$

follows by induction on F. At equations it is equational reasoning. At $Q_y B(v, y)$, use QC: transfer the new assumption $B(t, 0)$ backwards, apply the retained closure at Sz, and transfer the successor row forwards. At $D_{x,y}B(v, x, y)$, open the premise at fresh b, then introduce the target at b. At that point b is exactly the newest parameter. Transfer its base backwards, apply the opened closure, and transfer its successor row forwards. For a diagonal matrix in $\mathcal{J}$, its base is provable, so the same case construction supplies progress while ignoring the induction hypothesis.

This induction uses only transport for smaller formula templates. Its input families are regenerated whenever a fresh parameter is added.

3.2 Equation induction in the seven-rule version

For a term $s(y)$, define $L_s(y) = I(y, y; u.s(y))$. Unconditional cases, obtained from QC and QE, prove

$$C(y, s(0), L_s(y)) = s(y), \qquad C(v, a, b) := I(v, a; u.b). \tag{Dec}$$

At 0 this is the zero selector equation; at Sz it follows from two successor computations.

Given the step $s(u) = t(u) \vdash s(Su) = t(Su)$, retain $s(0) = t(0)$. Under $L_s(u) = L_t(u)$, the decoders and the retained base give $s(u) = t(u)$. Apply the step and compute back to $L_s(Su) = L_t(Su)$. This is a permitted diagonal introduction with an unused witness coordinate, so its deterministic choice is irrelevant. Elimination gives the conditional closure of the lift, whose base is reflexive. QE at Sz gives $s(Sz) = t(Sz)$ under the retained original base. QC now yields $Q_y(s(y) = t(y))$.

Thus $\mathcal{J}$, as well as $\mathcal{F}$, has portable induction for every literal equation. The argument uses QC, computations and equational reasoning, without a predecessor or an unrestricted witness.

4 The noncircular naming construction

4.1 Predecessor before existential naming

Appendix A proves, from computation, equational reasoning, and literal-equation induction alone, that a scalar iterator term pred satisfies

$$\mathrm{pred}(0) = 0, \qquad \mathrm{pred}(Sn) = n. \tag{P}$$

Hence $Sa = Sb$ implies $a = b$ before any arbitrary term has been named.

This dependency matters. The existing predecessor proof uses contextual parity cases in its Lean organization. Simply importing coverage into the restricted calculus would risk assuming the unrestricted introductions being recovered. The appendix replaces those steps by unconditional equational identities. Its central repair is

$$M(r + k) = I(k, M(r); u.c(u))$$

together with Boolean interpolation

$$H(Z(Zy)) = C(y, H(0), H(1)).$$

Every induction in that construction has a literal equation as its invariant.

4.2 A gate and a uniformly available witness

For an equation $E \equiv (s = t)$, write

$$G_v(E) := C(v, 0, s) = C(v, 0, t).$$

Its zero row is provable and its successor rows are interderivable with E, by computation and equational reasoning. Define

$$H(y) := D_{x,v}\, G_v(C(y, 0, Sx) = y). \tag{H}$$

First prove $H(0)$. Whatever witness ν_Π is selected, the inner equation $C(0, 0, S\nu_\Pi) = 0$ computes to $0 = 0$. The successor-row proof ignores its base assumption.

Next prove $H(Sz)$ in a scope whose newest parameter is z. The rule selects z, and $C(Sz, 0, Sz) = Sz$ is again a computation instance. The case parameter is appended after all surrounding parameters.

In $\mathcal{J}$ these matrices are always in case mode: their outer iterators have seed 0, whereas the diagonal guard requires seed v. QC gives $Q_y H(y)$, and QE with $H(0)$ gives

$$\Gamma \vdash H(St) \quad \text{for every term } t. \tag{HN}$$

This is an explicit QC construction followed by QE, not unrestricted generalization on a completed proof. The complete $H(St)$ construction is portable because its case parameter is selected anew in each outer scope.

Open $H(St)$ at fresh b. The new assumption is

$$Q_v G_v(C(St, 0, Sb) = St).$$

Its zero row is a theorem. QE at $1 = S0$, followed by computations, gives $Sb = St$. Applying pred and (P) gives

$$b = t. \tag{Alias}$$

This extraction uses only the retained closure, QE and equational reasoning, so the alias equation is portable while b is retained in scope.

One can also obtain a standalone naming formula

$$N_t := D_{x,v} G_v(x = t).$$

While $b = t$ is available and b is newest, introduce N_t, then discharge the opening of $H(St)$. This names t without supplying it as an introduction witness.

The gate cannot be omitted: $D_{x,v}(x = t)$ with v unused has the vacuous reading $\exists x((x = t) \to (x = t))$.

5 Completeness by regeneration of premises

Take a finite source derivation π, a capture-free substitution σ sending its ambient parameters to terms in a target scope, and portable realizations of its context after σ. We construct a portable target proof of its conclusion after σ.

Hypothesis, computation, term transport, and QE translate pointwise. At DE, produce the translated major, append a fresh target parameter b, and invoke the induction hypothesis on the source branch, extending σ to send its eigenparameter to b. The new closure assumption has a portable realization by Hyp. The conclusion is free of b, so target DE discharges it.

At an introduction with source witness w, put $t = \sigma(w)$. Construct $H(St)$ and open it at a new b, obtaining $b = t$. The target introduction must and does select b. Append its fresh step parameter z.

For a source case introduction, the target premise supplies $B(b, 0)$. Full formula transport gives $B(t, 0)$. Use this as the realization of the source branch's extra assumption when translating that *strictly smaller* source derivation. Its result $B(t, Sz)$ is transported back to $B(b, Sz)$. The target introduction is legal, and the outer DE discharges b. All additional assumptions remain in the target context; there is no operation substituting t into a completed target proof.

In the eight-rule translation, source QI uses target QI. For the seven-rule translation, start with a derivation in the existing diagonal calculus. Its guard is preserved and reflected by capture-free substitution of free parameters: such substitution cannot manufacture a bound occurrence of y. Thus source and target modes agree. In progress mode the same construction transfers $B(b, z)$ to $B(t, z)$, applies the recursively translated step, and transfers back. In case mode it is the preceding construction. The alias changes the selected witness, not the raw bound-variable guard.

For portability, run this procedure in each requested extension. No completed target derivation is copied through a new parameter scope. Recursive source calls decrease derivation height; auxiliary full transport decreases formula depth; naming and predecessor are fixed finite macros. This gives a terminating proof translation.

Conversely every target primitive inference is an instance of its source rule. The existing equivalence of the diagonal calculus with the native kernel [2] completes Theorem 1. Only *after* this result do we inherit arbitrary substitution, weakening and cut as admissible target rules. Direct copying of proof trees was not their proof of admissibility.

6 Why the interpretation requirement is met

The project proves faithful equivalence of the native calculus with its ordinary scalar realization T_{eq}, and a bi-interpretation of T_{eq} with iPRA [1, 3]. Both numerical domains are whole. An iterator translates to primitive recursion with the counter ignored in its step; the reverse translation compiles primitive recursive definitions into scalar iterators. The project proves term and formula round trips, not merely equiconsistency.

Our change gives an equal proof relation on the same terms, formulas and contexts. The existing translations and all round-trip proofs therefore remain valid. If τ, ρ are the project's translations, its native proofs of

$$A \dashv\vdash \rho(\tau(A))$$

translate by Theorem 1; the other composite is unchanged in iPRA. The domain predicates and identity graph $x = y$ are unchanged. This proves Corollary 2.

The artifact does not claim to re-run or extend the repository's Lean build. It uses the pinned existing bi-interpretation theorem and gives a written proof of the new identity comparison. Selected source files and exact URLs are included to make that dependency auditable.

7 Standalone checker and measured simplification

The main executable `code/checker_seven.py` implements $\mathcal{J}$; `code/checker_eight.py` implements $\mathcal{F}$. Each is a complete Python module using only the standard `json` library. It checks the input sequent's syntax and scope, then the proof, and prints `ACCEPT` only on success. Neither checker has a term normalizer or imports an arithmetic prover.

The syntax uses 0 for zero, negative integers $-1, \ldots, -n$ for ambient parameters, positive de Bruijn indices for bound variables, `[0,t]` for successor, and `[c,a,h]` for iteration. Formulas are `[s,t]`, `['q',B]`, and `[B]`. Input fields are `variables`, `context`, `goal`, and `proof`. A deterministic DI instruction is `['di', matrix, premise]`; its witness is $-n$, which is zero in empty scope.

Matched controls use the same representation, validators, other rules and I/O. Their DI instruction additionally supplies and validates an arbitrary term. The code reduction is directly tied to the mathematical restriction.

Complete checker	Tokens	AST nodes	Normalized bytes
Eight-rule control	979	1086	2834
Eight-rule deterministic witness	971	1078	2817
Seven-rule diagonal control	932	1029	2770
Seven-rule deterministic witness	924	1021	2753

All four delivered programs are the complete measured normalized sources, without omitted runtime helpers. The metric script uses CPython 3.12.14, `ast.unparse`, significant lexical tokens, and a full AST walk. Bytes include one final newline.

The reduction is modest: eight tokens, eight AST nodes, and seventeen bytes in each matched pair. In the pinned tagged-syntax programs the same direct change saves three bytes and one AST

node but adds two tokens, because constructing the selected tagged variable costs code. Those separate comparisons are recorded in the evidence.

The compact coordinate frontier—256 tokens, 278 AST nodes, 516 bytes— comes from different programs with different syntax contracts. Our standalone raw Eq/Q/D checker does not beat it. Seven rules also do not beat the existing three-family clause inventory. The improvement proved here is elimination of the introduction witness argument, with a matched complete-checker reduction and an equivalence on unrestricted native sequents.

8 Validation and exact limits

Executable validation covers all rule families, nontrivial naming scaffolding, malformed syntax, scoping, eigenparameter escape, and attempted false equations. Deterministic certificates are translated by inserting the selected witnesses and checked by matched controls and the pinned repository checkers. Results are recorded in `evidence/validation.json`.

The certificate for $H(2)$ obtains an existential successor witness through QC and QE even though no DI instruction carries an arbitrary term. It checks the numerical-cases portion of naming, not the entire predecessor construction or the metatheoretic equivalence.

Accordingly:

- The rule systems and new equivalence are mathematical claims with written proofs.
- Checker sizes and the included certificate results are reproducible executable evidence.
- No new Lean or Rocq formalization, exhaustive proof of Python correctness, absolute minimum, or global byte record is claimed.

Primitive equational transport is retained. This proof cannot simply be transferred to a three-family calculus whose transport itself is recovered from packets introduced at arbitrary witnesses; that would introduce a circular dependency.

9 Further research questions

1. Formalize the deterministic rules and the proof translation in Lean. Source derivations with realized contexts are the required induction interface; naively renaming target proof trees is insufficient.
2. Formalize the coverage-free predecessor as an equational-fragment theorem, separately from full numerical coverage.
3. Can deterministic introduction coexist with the three-family Name or inferred-opening calculus without circularly invoking unrestricted packet introduction to recover transport?
4. Does this restriction improve the actual compact 256-token, 278-node or 516-byte records under matched contracts?
5. Quantify certificate growth: replacing a witness term by a naming derivation makes the checker smaller but can make certificates larger.
6. Can scope determine witnesses while introduction modes also avoid a matrix guard, with a direct proof of all native consequences?

A Predecessor from equational induction alone

This section removes a possible circularity from the restriction of existential witnesses. The predecessor term is constructed before any general term-naming principle, and without numerical coverage, a separation axiom, existential introduction, or a case split under a hypothesis such as $c = 0$.

The equational interface. Use the term constructors 0, S, and $I(n, a; u.h)$, the two computation equations

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

ordinary equational reasoning, and induction for a literal equation. Thus, if z is absent from the parameters and hypotheses Γ, then

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}.$$

The unconditional cases rule

$$\frac{\Gamma \vdash E(0) \qquad \Gamma \vdash E(Sz)}{\Gamma \vdash E(t)}$$

is the special case that ignores the induction hypothesis. This rule does *not* replace or specialize hypotheses of Γ containing an arbitrary term c. Every use of cases below proves an unconditional identity with the displayed case variable and the other parameters independent.

All definitions below are abbreviations for scalar iterator terms. Write

$$C(c, a, b) = I(c, a; u.b), \qquad x + y = I(y, x; u.Su), \qquad 1 = S0.$$

Then $C(0, a, b) = a$, $C(Sz, a, b) = b$, $x + 0 = x$, and $x + Sy = S(x + y)$. Equation induction gives $0 + x = x$, $Sx + y = S(x + y)$, associativity and commutativity of addition. For example the second identity is induction on y, commutativity is induction on y using that identity, and associativity is induction on its last argument. Hence finite sums can be rearranged, and $Sx = x + 1$. The notation $2x$ and $3x$ means $x + x$ and $x + x + x$.

Zero test and parity. Put

$$Z(x) = C(x, 1, 0), \qquad P(n) = I(n, 0; u.Z(u)).$$

The equations $Z(0) = 1$, $Z(Sx) = 0$, and

$$Z(Z(Zx)) = Zx$$

follow by computation and unconditional cases on x. Consequently

$$P(0) = 0, \quad P(Sn) = Z(Pn), \quad Z(Z(Pn)) = Pn.$$

The last equation is induction on n: the base computes, and the step is the displayed triple-zero-test identity. It follows that $P(n + 2) = Pn$. Induction on n now gives

$$P(2n) = 0, \qquad P(3n) = Pn.$$

For the first step use $2(Sn) = 2n + 2$; for the second use $3(Sn) = 3n + 3$ and $P(m + 3) = Z(Pm)$.

Boolean interpolation without conditional cases. For any term template $H(v)$, with arbitrary other parameters, unconditional cases on y prove

$$H(Z(Zy)) = C(y, H(0), H(1)). \tag{BI}$$

Indeed $Z(Z0) = 0$ and $Z(Z(Sz)) = 1$. Thus if an equation $p = Z(Zp)$ is already available, equational congruence turns (BI) into $H(p) = C(p, H(0), H(1))$. In particular an identity between two term templates at 0 and at 1 holds at such a p. This uses a previously proved unconditional identity; it never substitutes into a contextual hypothesis.

The three-cycle and the ramp. Define

$$c(u) = Z(u) + 2P(u), \qquad M(n) = I(n, 0; u.c(u)), \qquad J(k, a) = I(k, a; u.c(u)).$$

Computation gives $c(0) = 1$, $c(1) = 2$, and $c(2) = 0$. The iterator equations give $M(0) = 0$, $M(Sn) = c(Mn)$, and induction on k gives the addition law

$$M(r + k) = J(k, M(r)). \tag{M+}$$

Its step is $M(r + Sk) = c(M(r + k)) = c(J(k, Mr)) = J(Sk, Mr)$.

Now put

$$R(n) = I(n, 0; u.S(u + M(u))).$$

We claim $M(Rn) = Pn$. For its induction step write $r = Rn$ and $p = Pn$, and use the induction hypothesis $Mr = p$. By (M+),

$$M(R(Sn)) = c(M(r + Mr)) = c(J(Mr, Mr)) = c(J(p, p)).$$

For a Boolean p, the identity

$$c(J(p, p)) = Zp$$

follows from (BI): at $p = 0$ both sides are 1, and at $p = 1$ both sides are 0, since $c(J(1, 1)) = c(c(1)) = c(2) = 0$. The already proved equation $p = Z(Zp)$ applies. This proves the induction step because $P(Sn) = Zp$. The base is immediate. We therefore have the simpler ramp recurrence

$$R(0) = 0, \qquad R(Sn) = Rn + Pn + 1. \tag{R}$$

Induction now gives

$$R(2n) = 3n, \qquad R(3n) = 3Rn + Pn. \tag{R23}$$

For the first step, $P(2n) = 0$ and $P(2n+1) = 1$, so two applications of (R) increase $R(2n)$ by 3. For the second, put $r = Rn$, $p = Pn$, and $q = Zp$. The three successive parities at $3n$, $3n + 1$, $3n + 2$ are p, q, p. Using the induction hypothesis, the next left side is therefore

$$R(3n + 3) = 3r + p + 3 + p + q + p = 3r + 3p + q + 3,$$

which is $3R(Sn) + P(Sn) = 3(r + p + 1) + q$. No conditional parity split is used.

A factor map. Let $H(x) = C(Px, Rx, x)$. Then

$$H(2x) = 3x, \qquad H(1) = 1, \qquad H(3x) = 3H(x).$$

The first two follow immediately. For the third put $p = Px$, $r = Rx$ and use (R23) to obtain

$$H(3x) = C(p, 3r + p, 3x) = C(p, 3r, 3x) = 3C(p, r, x).$$

The two identities used here,

$$C(v, a + v, b) = C(v, a, b), \qquad C(v, 3a, 3b) = 3C(v, a, b),$$

are unconditional identities proved by cases on v. In particular this argument also avoids conditional parity cases.

Comparison. Define

$$F(y, a) = I(y, a; u.H(u)), \qquad W(x) = I(x, 1; u.u + u), \qquad K(x, y) = P(F(y, W(x))).$$

Three elementary inductions on y give

$$H(F(y, a)) = F(y, H(a)), \qquad F(y, 3a) = 3F(y, a), \qquad F(y, 1) = 1.$$

Their steps use the iterator equation and, respectively, the preceding induction hypothesis, $H(3u) = 3H(u)$, and $H(1) = 1$. Since $W(0) = 1$ and $W(Sx) = 2W(x)$, we get

$$\begin{aligned} K(0, y) &= 1, \\ K(Sx, 0) &= 0, \\ K(Sx, Sy) &= P(H(F(y, 2W(x)))) \\ &= P(F(y, H(2W(x)))) \\ &= P(F(y, 3W(x))) = P(3F(y, W(x))) = K(x, y). \end{aligned}$$

Consequently $K(Sx, x) = 0$ by induction on x. The intended reading of $K(x, y)$ is the Boolean indicator of $x \leq y$, but that semantic reading is not needed: only the three displayed equations and their diagonal consequence are used below.

Saturation and predecessor. Put

$$B(k, u) = C(K(S(Su), k), u, Su), \qquad T(t, k) = I(t, 0; u.B(k, u)).$$

The comparison equations and the unconditional selector identity $C(v, Sa, Sb) = S(C(v, a, b))$ give

$$B(Sk, Su) = S(B(k, u)), \qquad B(S(Sk), 0) = 1.$$

Induction on t proves the conjugacy

$$T(St, S(Sk)) = S(T(t, Sk)). \tag{T}$$

At $t = 0$ it says $B(S(Sk), 0) = 1$. For the step, unfold the left side and rewrite its inner value by the induction hypothesis:

$$T(S(St), S(Sk)) = B(S(Sk), T(St, S(Sk))) = B(S(Sk), S(T(t, Sk))) = S(T(St, Sk)).$$

An induction on n, applying (T) at $t = k = n$ in its step, now gives $T(n, Sn) = n$.

Finally define the scalar iterator term

$$\mathrm{pred}(n) = T(n, n).$$

Its base equation is immediate, and its successor equation is

$$\begin{aligned}\mathrm{pred}(Sn) &= B(Sn, T(n, Sn)) = B(Sn, n)\\ &= C(K(S(Sn), Sn), n, Sn) = C(K(Sn, n), n, Sn) = C(0, n, Sn) = n.\end{aligned}$$

Therefore

$$\boxed{\mathrm{pred}(0) = 0, \qquad \mathrm{pred}(Sn) = n.}$$

Applying this term context to $Sa = Sb$ proves $a = b$.

Dependency conclusion. Every induction above has a literal equation as its invariant. Every case argument proves an unconditional equation and is itself an equation induction that ignores its induction hypothesis. The construction therefore belongs to the purely equational iterator fragment. It does not use the full numerical coverage principle, contextual case splitting, formula induction, successor injectivity, or a name for an arbitrary term. In a restricted native calculus with the two iterator equations, equational transport, QE and direct QI-progress, this entire fragment is available before DI is considered at all. This is the needed noncircular predecessor lemma for the witness restriction.

References

[1] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, README and research index, ProveIt, pinned commit `9f9545f5ffd9e0392d4a36639503aca12383450a`.
Pinned project and README

[2] ProveIt, *NativeDiagonal.lean*, same commit. Defines the diagonal calculus and proves `gprf_iff_nprf`.
Pinned diagonal calculus and equivalence proof

[3] ProveIt, *NativeBiInterpretation.lean*, same commit. Native/iPRA faithfulness, both formula round trips, and inconsistency equivalence.
Pinned native/iPRA bi-interpretation